

JEDI: Joint Embedding Diffusion World Model for Online Model-Based Reinforcement Learning

Jing Yu Lim Rushi Shah Zarif Ikram Samson Yu Haozhe Ma Tze-Yun Leong

Dianbo Liu

National University of Singapore

{jing.yu, shah.15, leongty, dianbo}@nus.edu.sg, zikram@usc.edu,
{samson.yu, haozhe.ma}@u.nus.edu.sg

Abstract

Diffusion world models have recently become competitive for online model-based reinforcement learning, but current approaches expose a tension: pixel diffusion is effective but computationally expensive while the latest latent diffusion approach improves efficiency yet performs subpar. The latter also relies on separately trained latents rather than the end-to-end world-model objectives that have driven much of modern MBRL progress. In particular, JEPA-style predictive representation learning has emerged as an especially promising direction for world modeling and MBRL. Concurrently, diffusion-style objectives have gained traction across multiple domains, with iterative refinement as a promising approach for multimodal and stochastic targets. Taken together, these trends motivate Joint Embedding Diffusion (JEDI), the first online end-to-end latent diffusion world model. JEDI learns its latent space directly from the diffusion denoising loss with a JEPA framework, using denoising to learn and predict future latents rather than relying on reconstruction and pretrained models. We provide a theoretical motivation showing that conventional JEPA objectives induce a predictive information bottleneck, and that conditional diffusion denoising admits a closely related predictive-compression decomposition. Empirically, JEDI is competitive on Atari100k and outperforms the baseline with separately trained latents where directly comparable. Relative to the pixel diffusion baseline, JEDI uses 43% less VRAM, over $3\times$ faster world-model sampling, and $2.5\times$ faster training. JEDI also exhibits a markedly different task-level performance profile from the pixel baseline, suggesting that end-to-end predictive latents change more than compute alone.

1 Introduction

Model-based reinforcement learning (MBRL) uses a learned world model [1, 2] to predict future outcomes and improve sample efficiency for training an online RL agent [3]. Diffusion models have recently emerged as a powerful backbone for world modeling, especially in visually complex domains [4–6]. However, the strongest online diffusion world models for MBRL, DIAMOND, still operate in pixel space, where denoising is expensive in memory, sampling time, and training time [7]. A natural next step is therefore to move diffusion into latent space.

Recent latent diffusion approaches such as Horizon Imagination (HI) [8] show that this can improve efficiency, but they learn their latents without end-to-end sequential prediction and do not yet match the performance of DIAMOND [7]. This leaves a clear opportunity: if much of the past progress in

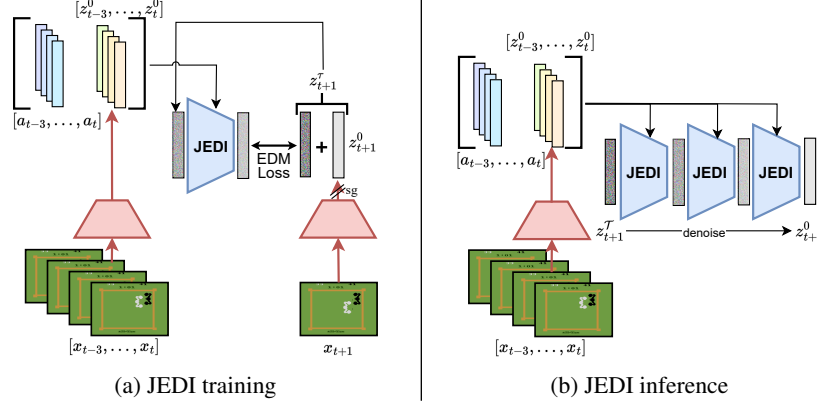


Figure 1: Joint Embedding Diffusion (JEDI) world model. During training, observations are encoded into clean latents, the next latent is noised, and the denoiser predicts the direction back toward the clean future latent while gradients update the encoder end to end. During inference, the model conditions on past latents and actions, starts from noise, and denoises to the next latent for imagination and policy learning.

MBRL has come from end-to-end representation learning with world-model objectives, then latent diffusion may benefit from the same principle [9, 3, 10–13].

One especially promising way to instantiate the principle of end-to-end predictive representation learning is through Joint Embedding Predictive Architecture (JEPA). The goal is to learn representations that capture the predictable, abstract structure of the world while avoiding the need to reconstruct every low-level or intrinsically unpredictable detail [14–18]. That inductive bias is especially appealing for MBRL, where the latent should retain information useful for future prediction, rewards, and control, while discarding nuisance variation that is irrelevant for decision making [19–23].

In parallel, there has been a general shift toward diffusion-style objectives rather than one-shot prediction across several domains. These include policy learning [24–26], language modeling [27–29], and even classification [30]. These works suggest that diffusion is especially useful for multimodal, stochastic, or sequential targets, where iterative denoising can support more expressive prediction and greater robustness to uncertainty. In the context of MBRL, this makes diffusion a relevant objective to examine for end-to-end latent world models.

Taken together, these observations raise a natural question: can a diffusion world model learn its latent representation end to end from the denoising objective with JEPA? We answer yes. We introduce Joint Embedding Diffusion (JEDI), a latent diffusion world model that trains its encoder end-to-end through conditional denoising, together with reward and end prediction (fig. 1). To our knowledge, JEDI is the first online end-to-end latent diffusion world model built on a JEPA framework.

We show that this representation-learning strategy is theoretically motivated (section 2): latent conditional diffusion admits a variational information-bottleneck decomposition [31–33], helping explain why latent conditional denoising can support end-to-end latent learning in a world model.

Empirically, JEDI delivers strong results on full Atari100k [34] and in supporting Craftium [35] experiments (figs. 4 to 6) while dramatically improving efficiency over DIAMOND (fig. 8 and table 1), using 43% less VRAM, more than $3\times$ faster world-model sampling, and more than $2.5\times$ faster training.

More broadly, these results suggest that *predictive* latent diffusion is a promising alternative to latents trained separately with reconstruction and perceptual losses for online MBRL. While DIAMOND remains a strong baseline, our results suggest that end-to-end predictive latents may offer a more attractive path forward when both efficiency and performance matter. Finally, we also observe a markedly different performance profile across games relative to DIAMOND (figs. 11 and 15), suggesting that end-to-end predictive latents change more than compute alone (section 5).

2 Theoretical Motivation: Information Bottleneck in JEPA

We show that JEPA can be given a variational information-bottleneck interpretation [33, 32, 31] complementary to prior formulations for self-supervised representation learning [36, 37]. With the Probabilistic Graphical Model (PGM) in fig. 2, the bottleneck structure emerges directly from the JEPA predictive objective:

$$\mathcal{L}_{\text{JEPA}} := \mathbb{E}_{p(x_1, x_2) q_\phi(z_1 | x_1)} [D_{\text{KL}}(q_\phi(z_2 | x_2) \| p_\theta(z_2 | z_1))]. \quad (1)$$

Here, x_1 and x_2 can denote sequential observations in an MDP, with the action omitted for clarity, or two views in a multiview SSL setting. In practice, this KL reduces to cross-entropy for categorical targets and to mean-squared error for fixed-variance Gaussian targets, corresponding to existing works [38, 17]. This KL term is also common in canonical end-to-end MBRL works [3, 10, 11, 19, 20]. Appendix A makes this precise and shows that the JEPA objective can be written as

$$-\mathcal{L}_{\text{JEPA}} = I(X_1; X_2) - \hat{I}(X_1; Z_1) - \hat{I}(X_2; Z_2) + \mathcal{R}_1 - \mathcal{G}, \quad (2)$$

where $\hat{I}(X_i; Z_i)$ are variational mutual-information estimates, $\mathcal{R}_1$ is the bottleneck regularizer, and $\mathcal{G}$ is the posterior approximation gap. Using the data processing inequality yields the bound:

$$-\mathcal{L}_{\text{JEPA}} \leq I(Z_1; Z_2) - \hat{I}(X_1; Z_1) - \hat{I}(X_2; Z_2) + \mathcal{R}_1 - \mathcal{G}. \quad (3)$$

The decomposition makes an information-bottleneck structure explicit. More precisely, it shows that the predictive JEPA loss appears inside a variational bottleneck objective (recovering that full objective would additionally require explicit optimization of $\mathcal{R}_1$). Nonetheless, if we treat Z_2 as the target label and Z_1 as the learned representation, then the JEPA objective mirrors the deep variational information bottleneck [31]: it rewards representations that retain information useful for predicting the target through the $I(Z_1; Z_2)$ term, while penalizing information retained about the inputs through the compression terms. In our setting, this compression naturally appears symmetrically across the two input branches, through $\hat{I}(X_1; Z_1)$ and $\hat{I}(X_2; Z_2)$.

2.1 Information Bottleneck in JEDI’s Latent Conditional Diffusion

The same perspective extends to latent conditional diffusion. This connection is especially natural from the PGM perspective underlying DDPM [39], extended to the latent space (fig. 3). In JEDI, one-step latent prediction is replaced by a latent conditional denoising process that predicts the clean future latent (z_2^0) while conditioning on the current latent (z_1^0) and noised future latent (z_2^t). Following standard DDPM training, we optimize a conditional denoising objective consisting of an endpoint loss term $\mathcal{L}^0$ and denoising KLs:

$$\mathcal{L}_{\text{CDJ}}^{\text{den}} := \mathcal{L}^0 + \mathbb{E}_{p(x_1, x_2)} \sum_{t=2}^T \mathbb{E}_{q_\phi(z_1^0 | x_1) q_\phi(z_2^0 | x_2) q(z_2^t | z_2^0)} D_{\text{KL}}(q(z_2^{t-1} | z_2^t, z_2^0) \| p_\psi(z_2^{t-1} | z_2^t, z_1^0)) \quad (4)$$

where the loss is typically implemented as regressing towards sampled noise [39]. This yields an analogous information-bottleneck structure (Appendix A.2):

$$-\mathcal{L}_{\text{CDJ}}^{\text{den}} \leq I(Z_1^0; Z_2^0) - \hat{I}(X_1; Z_1^0) - \hat{I}(X_2; Z_2^0) + \mathcal{R}_1 + \mathcal{R}_{2,T} - \mathcal{G} + \mathcal{C}_2, \quad (5)$$

where $\mathcal{R}_{2,T}$ is the terminal prior-matching term, and $\mathcal{C}_2$ is a constant with deterministic encoders. In our current implementation, we optimize this denoising term rather than the full objective augmented with $\mathcal{R}_1$. This perspective also points to a natural extension: explicitly regularizing the bottleneck term, potentially in ways related to recent collapse-prevention methods such as VICReg and LeJEPA [40, 41] (section A.2.1).

This is the central mathematical motivation for JEDI: latent conditional denoising is not merely a generative surrogate, but supplies the predictive term in a JEPA-style variational bottleneck decomposition. Accordingly, our claim is that diffusion denoising provides a principled end-to-end predictive learning signal for the latent encoder, rather than that the method exactly optimizes the complete variational objective.

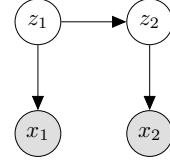


Figure 2: JEPA PGM

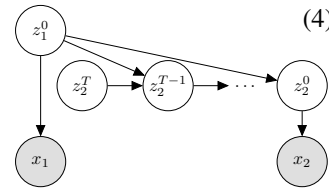


Figure 3: Latent conditional diffusion PGM. The clean latent z_1^0 acts as the context representation, and the reverse diffusion chain predicts the clean target latent z_2^0 through a denoising trajectory.

3 Method

3.1 JEDI: Joint Embedding Diffusion World Model

We introduce the Joint Embedding Diffusion World Model (JEDI), which learns an end-to-end latent space encoder while leveraging a diffusion dynamics model. The encoder compresses the observation into a clean latent z_t^0 , the dynamics model predicts the next latent through conditional denoising, and reward and termination are predicted from the latent state. The world model is

$$\text{World Model: } \begin{cases} \text{Encoder:} & z_t^0 = \mathbf{E}_\phi(x_t), \\ \text{Latent diffusion dynamics:} & \hat{z}_{t+1}^0 \sim \mathbf{S}(\mathbf{D}_\theta(\hat{z}_{t+1}^\tau, Z_t^\tau)), \\ \text{Reward and termination:} & (\hat{r}_t, \hat{d}_t) = \mathbf{R}_\psi(z_t^0), \end{cases} \quad (6)$$

where t is the environment time step, $x_t \in [0, 1]^{64 \times 64 \times 3}$ is the observation, $z_t^0 \in [-3, 3]^{16 \times 8 \times 8}$ is the clean latent, and τ is the diffusion time step sampled from a log-normal distribution that determines the noise scale. We write $Z_t^\tau := (c_{\text{noise}}^\tau, z_{t-3:t}^0, a_{t-3:t})$ for the conditioning tuple consisting of the diffusion-time embedding, past latent states, and past actions. As in EDM [42], we use the preconditioned denoiser

$$\mathbf{D}_\theta(z_{t+1}^\tau, Z_t^\tau) = c_{\text{skip}}^\tau z_{t+1}^\tau + c_{\text{out}}^\tau \mathbf{F}_\theta(c_{\text{in}}^\tau z_{t+1}^\tau, Z_t^\tau). \quad (7)$$

Here c_{noise}^τ is a fixed transformation of τ , while c_{skip}^τ , c_{out}^τ , and c_{in}^τ are the standard EDM preconditioning coefficients that improve the behavior of the neural network $\mathbf{F}_\theta$. Training and inference follow the same conditioning pipeline shown in fig. 1. During training, the encoder first derives $z_{t-3:t+1}^0$ from observations $x_{t-3:t+1}$. The future latent z_{t+1}^τ is then obtained by perturbing z_{t+1}^0 with diffusion noise at level τ , and $\mathbf{F}_\theta$ is trained to predict the direction toward the clean next latent. During inference, the conditioning tuple is formed in the same way, except that the future latent is initialized from noise and the solver $\mathbf{S}$ iteratively denoises it to obtain $\hat{z}_{t+1}^0$.

The key training signal is the joint embedding diffusion loss

$$\mathbb{E}_{z_{1:T} \sim q, x_{1:T} \sim p, \tau \sim \text{LN}} \left[\left\| \sum_{t=1}^T \mathbf{F}_\theta(c_{\text{in}}^\tau \text{sg}(z_{t+1}^\tau), Z_t^\tau) - \frac{1}{c_{\text{out}}^\tau} (\text{sg}(z_{t+1}^0) - c_{\text{skip}}^\tau \text{sg}(z_{t+1}^\tau)) \right\|^2 \right]. \quad (8)$$

where $q(z_t)$ denotes the deterministic pushforward of the observation distribution through the encoder, and sg denotes stop-gradient. The reward and termination head is trained with

$$\mathbb{E}_{z_{1:T}^0 \sim q, (x_{1:T}, r_{1:T}, d_{1:T}) \sim p} \sum_{t=1}^T \text{CE}(\mathbf{R}_\psi(z_t^0), (r_t, d_t)). \quad (9)$$

Crucially, gradients from both losses update the encoder directly through z_t^0 , so the latent is learned through predictive world-model training rather than through reconstruction.

3.2 Practical Design Choices

Several design choices are important in practice. First, to reduce representation collapse, we apply stop-gradient to the future latent target and set the encoder learning rate to 0.3 times the denoiser learning rate, following the spirit of JEPA and TDMPC2 [17, 20]. This allows the encoder to learn directly from the denoising objective while preventing unstable feedback through the future latent target. Second, to stabilize iterative denoising in latent space, we clamp latent activations with the differentiable transform $C(z) = \tanh(z/s)s$ with $s = 3$, applied to both encoder outputs and denoiser outputs. This is common practice: DIAMOND clamps pixels to the natural $[-1, 1]$ image range and discretizes that range into 256 bins, while HI uses a plain $\tanh$ to clamp latents [7, 8]. Third, with every trajectory batch we randomly switch with uniform probability between using the denoiser output $\mathbf{D}_\theta$ and the encoder output $\mathbf{E}_\phi$ as the next-step conditioning signal. This balances near-horizon consistency with sufficient direct encoder supervision.

The policy is trained in an actor-critic framework [10, 7]. We intentionally use REINFORCE [43] so that the actor does not require backpropagation through multi-step denoising, which keeps policy learning lightweight despite the iterative diffusion world model.

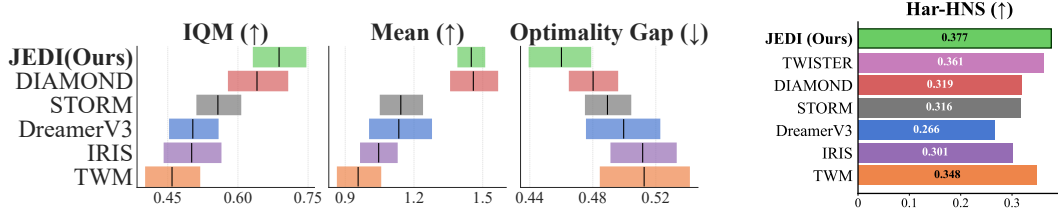


Figure 4: Aggregate Atari100k performance. Left: IQM, mean, and optimality gap following Agarwal et al. [44]. Right: Har-HNS (section 4) is shown as a complementary descriptive statistic because JEDI exhibits a different task-level profile from prior baselines. TWISTER does not report seed-level results, so mean, IQM and optimality-gap error bars are unavailable. Full per-game numbers are reported in Appendix table 5.

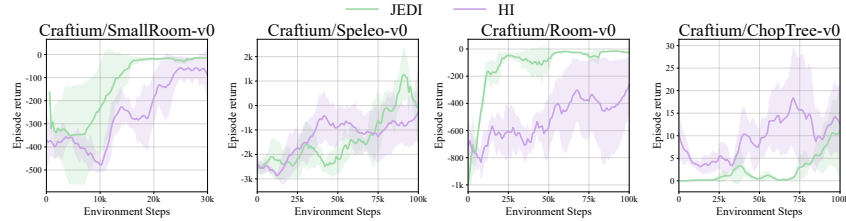


Figure 5: Craftium performance comparison across JEDI and HI

4 Experiments

Experimental Setup Our primary baselines are DIAMOND [7] and HI [8]. DIAMOND is the canonical online pixel-space diffusion world model and the cleanest comparison for the central pixel-versus-end-to-end-latent question. HI is the most relevant latent-diffusion comparison, since it shows that latent diffusion can be efficient but does not learn the latent space end to end through the predictive world-model objective. We also compare against DreamerV3, STORM¹, TWM, IRIS, and TWISTER when published numbers are available [11, 12, 45, 13, 46, 47].

We evaluate JEDI primarily on the full Atari100k benchmark with 26 tasks and 5 seeds per task [34]. Each run takes about two days on an A100. For each task, we report the better result from two diffusion-model batch sizes, 32 and 64. Following Agarwal et al. [44], we emphasize interquartile mean, optimality gap, performance profile of the Human-Normalized Scores (HNS), and we use bootstrap sampling for error bars when seed-level data are available. Refer to section B.1 for more details.

In addition to Atari100k, we evaluate on Craftium [35], a 3D embodied Minecraft-like decision-making environment that lets us test whether JEDI transfers beyond 2D Atari. For Craftium, we run 3 seeds per task. Craftium also gives the clearest direct empirical comparison to HI, since it reports published training-curve results there but not on the full Atari100k benchmark.

Har-HNS Arithmetic mean HNS is an important aggregate metric, but it is disproportionately influenced by already high-valued games and can therefore obscure large relative gains on low-HNS tasks [48, 49]. As a result, methods that improve low-HNS games can appear less impressive under arithmetic averaging than their task-level profile would suggest. Critically, although JEDI and DIAMOND have similar mean HNS, they improve in very different HNS regimes (figs. 11 and 15).

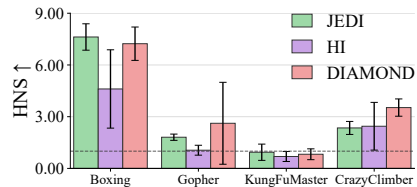


Figure 6: Atari100k performance comparison across JEDI, HI, and DIAMOND

¹We derive the latest updated STORM results from [45]

Inspired by the F1 score [50], the use of harmonic mean provides a complementary perspective by emphasizing improvements where performance is still poor. Formally, we define

$$\text{Har-HNS} = \left(\frac{1}{N} \sum_{i=1}^N \frac{1}{\text{HNS}_i + 0.1} \right)^{-1}, \quad (10)$$

where N is the number of games, HNS_i is the average HNS for game i , and the offset of 0.1 ensures numerical stability for negative HNS values. Intuitively, mean HNS asks how much total score we collect across tasks, whereas Har-HNS asks how well we are doing on the tasks that remain very difficult. We therefore use Har-HNS as a complement to mean HNS to better understand the changed performance profile induced by end-to-end latent diffusion.

4.1 Main Atari100k Results

As shown in fig. 4 and the full per-game breakdown in Appendix table 5, JEDI achieves strong Atari100k performance. It matches or exceeds strong baselines on IQM, optimality gap, and number of state-of-the-art game scores, while remaining competitive on arithmetic mean HNS and achieving runner-up performance in the number of superhuman tasks.

On the four Atari tasks reported by HI, JEDI also performs better overall on the shared subset (fig. 6). This sharpens the paper’s main empirical point. The contribution is not merely that latent diffusion can be cheaper than pixel diffusion, but that an end-to-end predictive denoising, trained directly through the world-model objective, is sufficient to support a competitive online diffusion world model without the need for a pretrained model for learning latents.

Stochastic Atari. All of Atari100k tasks are deterministic. To test whether diffusion helps model stochastic targets, we compare JEDI with DreamerV3, an MLP-based end-to-end latent world model, on three Atari environments with random frameskips between 2 and 6. JEDI consistently outperforms DreamerV3, suggesting that diffusion may be more robust to high task-related aleatoric uncertainty.



Figure 7: JEDI vs. DreamerV3 on Atari with random frameskip. The top row shows performance without frame-skipping, and the bottom row shows performance with stochastic frame-skipping.

4.2 Craftium Results

Craftium provides supporting evidence that JEDI is not specific to Atari100k. As shown in fig. 5, the same qualitative picture appears here as well: end-to-end predictive latents can support a strong latent diffusion world model without reconstruction-based latent training. In a direct comparison with HI on four Craftium tasks, JEDI performs better overall. Because Craftium is a 3D embodied Minecraft-like environment, these results also suggest that the approach extends beyond 2D Atari.

4.3 Efficiency and Scalability

As summarized in fig. 8, relative to DIAMOND, JEDI uses 43% less VRAM, achieves more than $3\times$ faster world-model sampling, and trains more than $2\times$ faster. These gains come from moving diffusion from an observation in $\mathbb{R}^{64\times64\times3}$ to a $12\times$ smaller $z_t^0 \in \mathbb{R}^{16\times8\times8}$ compact learned latent with end-to-end training. This is the clearest practical takeaway of the paper: predictive latent diffusion is much more scalable than pixel-space diffusion for online MBRL.

Method	A100 (hrs)	V100 (hrs)	#Params
JEDI	38	-	13.5M
HI [8]	27	-	97M
DIAMOND [7]	98	-	13.5M
DreamerV3 [11]	-	12	18M
STORM [12]	7	9.3	18.8M
TWM [13]	10	-	21.6M
IRIS [46]	168	-	30M

Table 1: Wall clock run times per Atari100k run on A100 / V100 and model size

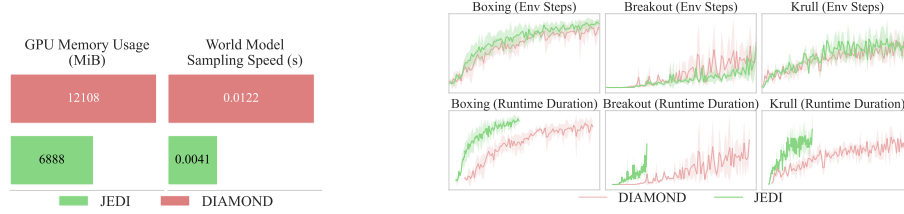


Figure 8: Left: JEDI uses 57% of DIAMOND’s GPU memory while sampling the world model over $3\times$ faster. Right: JEDI reaches comparable performance in less than half the wall-clock training time.

The most relevant efficiency comparison is DIAMOND: as shown in table 1, JEDI stays competitive while cutting A100 training time from 98 to 38 hours at the same parameter count. HI is faster, but JEDI better preserves latent diffusion’s efficiency advantage without giving up competitive end-to-end performance.

4.4 Ablation Study

We conduct two sets of ablation studies, each over five Atari tasks. The first set studies how the latent representation is learned, while the second isolates several stabilization and design choices.

Latent Learning ablation Figure 9 shows that how the latent space is learned matters. Full JEDI performs best. Replacing the denoising objective with direct next-state MSE prediction reduces performance, and removing diffusion-loss gradients hurts further. Using a separately trained VAE latent performs worst, while adding decoder gradients also slightly hurts performance.

Overall, these ablations support learning the latent space jointly with the predictive world model and point to the denoising objective as a key training signal.

Design Choice Ablation We also study the design choices that help make end-to-end latent diffusion stable and competitive (fig. 10). Variants using an EMA target encoder, deterministic switching, or no random switching perform worse than the default design, and removing switching together with latent clamping hurts further. Still, HNS remains high across these ablations, suggesting that these design choices are helpful rather than critical. The end-to-end predictive objective appears to be the main ingredient, while the stabilization techniques remain important supporting details.

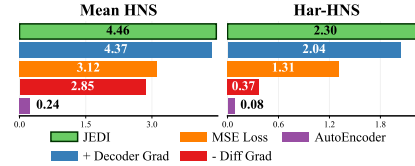


Figure 9: Latent-learning ablations on five Atari tasks. Joint predictive training outperforms adding decoder-based reconstruction supervision (+ Decoder Grad), substituting diffusion loss for MSE loss (MSE Loss), removing diffusion loss gradients (- Diff Grad) and using separately VAE trained latents (AutoEncoder).

5 Discussion

5.1 Performance Profile and Har-HNS

JEDI has a distinctly different performance profile from DIAMOND (fig. 11). Importantly, we build directly on top of DIAMOND and keep the experimental setup as close as possible, with the main change being the end-to-end JEPA latent space and only minor hyperparameter adjustments (section B). JEDI’s top quantile of relative gains occurs where DIAMOND’s HNS is very low (mean ≈ 0.06), whereas DIAMOND’s top quantile occurs in much higher-HNS regimes (mean ≈ 0.81), as shown in figs. 11 and 15.

This accounts for fig. 4 the aggregate ranking flips under Har-HNS (section 4): although JEDI (1.450) is slightly below DIAMOND (1.459) on arithmetic mean HNS (table 5), it attains the strongest

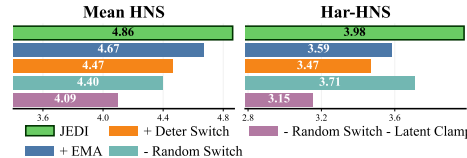


Figure 10: Design-choice ablations. Variants test EMA targets, deterministic switching, removing random switching, and removing switching together with latent clamping.

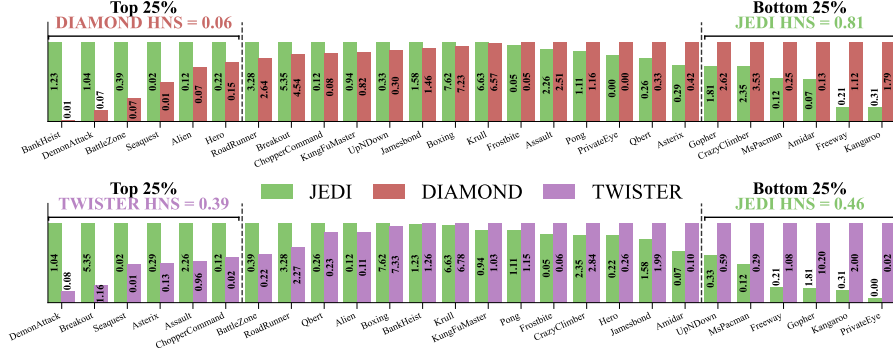


Figure 11: Task-level performance profiles for JEDI versus DIAMOND and TWISTER. Tasks are ordered by JEDI’s relative gain over each comparison method, with the top and bottom quartiles shown separately. Bar heights are normalized, while the annotated values report HNS. JEDI excels against DIAMOND on low-HNS tasks, whereas DIAMOND tends to outperform JEDI on high-HNS tasks. This distinct performance profile is less apparent when comparing JEDI with TWISTER, a latent baseline.

Har-HNS at 0.377, ahead of DIAMOND (0.319). This indicates that JEDI fails less severely on the games that remain hardest.

5.2 Why Does the Performance Profile Change?

A plausible explanation is that introducing an end-to-end latent interface changes which games are easiest for the policy to optimize. All of JEDI’s six top-quantile games have shooter-style dynamics with five of them having the maximum action space, motivating the aggregated breakdown in fig. 12 (see also table 4). As summarized in fig. 12, JEDI’s largest gains are concentrated in higher-complexity games, especially games with larger action spaces and shooter-style dynamics. DIAMOND, in contrast, feeds raw image features to its actor and critic, so under a fixed compute budget it must spend capacity on both representation learning and policy/value learning. JEDI instead gives the actor and critic a learned latent $z_t^0 \in \mathbb{R}^{16 \times 8 \times 8}$ rather than an observation in $\mathbb{R}^{64 \times 64 \times 3}$, making the actor-critic input about $12 \times$ smaller and reducing the need for those networks to solve representation learning themselves.

Intuitively, the effective interaction space between inputs and action/value prediction can grow combinatorially—often effectively exponentially—with input dimension, so this reduction may matter most in large-action, visually complex games. We regard this as an interpretation supported by the empirical pattern, not as a formal causal proof.

5.3 Qualitative Analysis

The changed task-level performance profile is also visible qualitatively (fig. 13). On *BankHeist*, *DemonAttack*, and *Hero*—all in JEDI’s top relative-performance quantile—JEDI learns visibly different policies from DIAMOND: it repeatedly re-enters maps in *BankHeist* to collect easy bank spawns while avoiding self-destructive *FIRE* actions, more reliably destroys obstacles in *Hero*, and actively tracks and neutralizes enemies in *DemonAttack* rather than staying in a corner and firing ineffectively.

We view these trajectories as consistent with the broader performance-profile story: moving to an end-to-end predictive latent may help more on complex games with large action spaces and shooter dynamics, where a compressed world-model latent may better align the policy-learning interface with downstream value estimation.

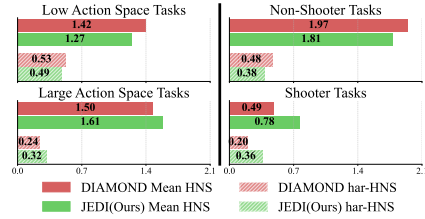


Figure 12: Task-property analysis comparing high versus low action-space games and shooter versus non-shooter games.

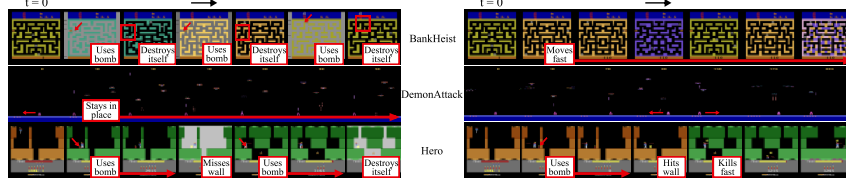


Figure 13: Example trajectories with DIAMOND (LEFT) and JEDI (RIGHT) on three tasks. JEDI is significantly more effective at eliminating enemies and obstacles while effectively minimizes self-destruction as compared to DIAMOND

Conclusion To our knowledge, JEDI is the first to show that diffusion world models can be trained in an end-to-end predictive latent space using JEPA-style learning. JEDI preserves strong online MBRL performance and substantially improving efficiency over pixel diffusion. Beyond the immediate compute gains, our results suggest that predictive latent learning changes the behavior of the world model itself: JEDI exhibits a markedly different task-level performance profile from pixel-space diffusion, indicating that end-to-end latent learning affects more than runtime and memory alone. Together with the information-bottleneck perspective developed in our theory section, these findings position predictive latent diffusion as a principled and practical direction for scaling online MBRL.

Limitations. Our evidence is limited to Atari100k (5 seeds) and four Craftium tasks (3 seeds; figs. 4 and 5), so we have not yet tested JEDI on DMControl, Procgen, or higher-resolution domains. JEDI is also not the fastest latent method: HI trains in 27 A100-hours versus our 38, though HI uses a pretrained perceptual model and 97M parameters, versus JEDI’s 13.5M (table 1). Theoretically, eqs. (1) to (5) motivate representation-learning properties rather than guaranteeing optimization. Performance also depends on several design choices, including tanh clamping and random switching (figs. 9 and 10); DIAMOND and HI also use clamping, and DIAMOND additionally uses latent discretization. Finally, performance is sensitive to denoiser batch size, so we report the better of batch sizes 32 and 64 for each task; HI similarly reports results across multiple hyperparameter settings.

6 Related Works

World models and online MBRL. Classic approaches such as Dyna and PlaNet established predictive world models for control [1, 9]. Search-based successors such as MuZero and EfficientZero are outside our scope because we focus on online MBRL without look-ahead planning [51–53]. Dreamer and its successors showed that actor-critic learning on imagined trajectories scales to visual domains [3, 10, 11]. IRIS, TWM, STORM, and TWISTER expanded the Atari100k design space with sequence-model world models [46, 13, 12, 47]. DIAMOND introduced strong pixel diffusion for online MBRL, while Horizon Imagination moved diffusion to a separately learned latent space [7, 8]. Other concurrent methods add annotations, event supervision, or explicit exploration [45, 54, 55] and are beyond our scope of studying end-to-end latent diffusion for MBRL world model backbone.

JEPA and predictive representations. Early work argued that representations should preserve predictive structure rather than reconstruct raw detail [14], a view further developed by modern JEPA-style methods [15, 38, 17, 18, 21]. Related predictive-latent methods also perform well in model-based control [19, 20, 23, 22]. Recent work further connects JEPA to the information bottleneck [56, 31, 33, 36, 40, 37]. To our knowledge, JEDI is the first online world model to combine a JEPA-style objective with diffusion denoising and an information-bottleneck interpretation.

Diffusion Diffusion models are now a standard generative framework [57, 39, 42]. Many high-capacity systems use separately learned latent spaces for efficiency [58–60, 5, 4]. LSGM also studied end-to-end latent generative modeling, but with reconstruction-based training [61]. Diffusion has also been effective in planning, control, and offline world modeling [62, 24, 63, 25, 26, 64, 65], and has been used for representation learning through denoising-step search, distillation, compressed conditioning, and hidden-feature extraction [66–73]. JEDI instead learns and predicts compressed latents end to end through a JEPA-style diffusion world-model objective.

References

- [1] Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- [2] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3), 2018.
- [3] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- [4] Yixin Liu, Kai Zhang, Yuan Li, Zhiling Yan, Chujie Gao, Ruoxi Chen, Zhengqing Yuan, Yue Huang, Hanchi Sun, Jianfeng Gao, et al. Sora: A review on background, technology, limitations, and opportunities of large vision models. *arXiv preprint arXiv:2402.17177*, 2024.
- [5] J Parker-Holder, P Ball, J Bruce, V Dasagi, K Holsheimer, C Kaplanis, A Moufarek, G Scully, J Shar, J Shi, et al. Genie 2: A large-scale foundation world model. URL: <https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model>, 2024.
- [6] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.
- [7] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos J Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37:58757–58791, 2024.
- [8] Lior Cohen, Ofir Nabati, Kaixin Wang, Navdeep Kumar, and Shie Mannor. Horizon imagination: Efficient on-policy rollout in diffusion world models. *arXiv preprint arXiv:2602.08032*, 2026.
- [9] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pages 2555–2565. PMLR, 2019.
- [10] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.
- [11] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [12] Weipu Zhang, Gang Wang, Jian Sun, Yetian Yuan, and Gao Huang. Storm: Efficient stochastic transformer based world models for reinforcement learning. *Advances in Neural Information Processing Systems*, 36:27147–27166, 2023.
- [13] Jan Robine, Marc Höftmann, Tobias Uelwer, and Stefan Harmeling. Transformer-based world models are happy with 100k interactions. *arXiv preprint arXiv:2303.07109*, 2023.
- [14] Jürgen Schmidhuber and Daniel Prelinger. Discovering predictable classifications. *Neural Computation*, 5(4):625–635, 1993.
- [15] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020.
- [16] Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- [17] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15619–15629, 2023.

- [18] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024.
- [19] Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.
- [20] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- [21] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohov, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [22] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim GJ Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. *arXiv preprint arXiv:2603.12231*, 2026.
- [23] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [24] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [25] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.
- [26] Kevin Frans, Seohong Park, Pieter Abbeel, and Sergey Levine. Diffusion guidance is a controllable policy improvement operator. *arXiv preprint arXiv:2505.23458*, 2025.
- [27] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- [28] Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion beats autoregressive in data-constrained settings. *arXiv preprint arXiv:2507.15857*, 2025.
- [29] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in neural information processing systems*, 35:4328–4343, 2022.
- [30] Omer Belhasin, Shelly Golan, Ran El-Yaniv, and Michael Elad. Advancing image classification with discrete diffusion classification modeling. *arXiv preprint arXiv:2511.20263*, 2025.
- [31] Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. Deep variational information bottleneck. *arXiv preprint arXiv:1612.00410*, 2016.
- [32] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- [33] Ravid Shwartz-Ziv and Naftali Tishby. Opening the black box of deep neural networks via information. *arXiv preprint arXiv:1703.00810*, 2017.
- [34] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, et al. Model-based reinforcement learning for atari. *arXiv preprint arXiv:1903.00374*, 2019.
- [35] Mikel Malagón, Josu Ceberio, and Jose A Lozano. Craftium: Bridging flexibility and efficiency for rich 3d single-and multi-agent environments. *arXiv preprint arXiv:2407.03969*, 2024.

- [36] Shiye Wang, Changsheng Li, Yanming Li, Ye Yuan, and Guoren Wang. Self-supervised information bottleneck for deep multi-view subspace clustering. *IEEE Transactions on Image Processing*, 32:1555–1567, 2023.
- [37] Ravid Shwartz Ziv and Yann LeCun. To compress or not to compress—self-supervised learning and information theory: A review. *Entropy*, 26(3):252, 2024.
- [38] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.
- [39] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [40] Ravid Shwartz-Ziv, Randall Balestriero, Kenji Kawaguchi, Tim GJ Rudner, and Yann LeCun. An information theory perspective on variance-invariance-covariance regularization. *Advances in neural information processing systems*, 36:33965–33998, 2023.
- [41] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- [42] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35: 26565–26577, 2022.
- [43] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- [44] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.
- [45] Weipu Zhang, Adam Jelley, Trevor McInroe, and Amos Storkey. Objects matter: object-centric world models improve reinforcement learning in visually complex environments. *arXiv preprint arXiv:2501.16443*, 2025.
- [46] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. *arXiv preprint arXiv:2209.00588*, 2022.
- [47] Maxime Burchi and Radu Timofte. Learning transformer-based world models with contrastive predictive coding. *arXiv preprint arXiv:2503.04416*, 2025.
- [48] Philip J Fleming and John J Wallace. How not to lie with statistics: the correct way to summarize benchmark results. *Communications of the ACM*, 29(3):218–221, 1986.
- [49] Joint Research Centre. *Handbook on constructing composite indicators: methodology and user guide*. OECD publishing, 2008.
- [50] C Van Rijsbergen. Information retrieval: theory and practice. In *Proceedings of the joint IBM/University of Newcastle upon tyne seminar on data base systems*, volume 79, pages 1–14. Butterworth-Heinemann Oxford, UK, 1979.
- [51] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- [52] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- [53] Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. *Advances in neural information processing systems*, 34:25476–25488, 2021.

- [54] Zhao-Han Peng, Shaohui Li, Zhi Li, Shulan Ruan, Yu Liu, and You He. From observations to events: Event-aware world model for reinforcement learning. *arXiv preprint arXiv:2601.19336*, 2026.
- [55] Lior Cohen, Kaixin Wang, Bingyi Kang, Uri Gadot, and Shie Mannor. Uncovering untapped potential in sample-efficient world model agents. *arXiv preprint arXiv:2502.11537*, 2025.
- [56] Naftali Tishby, Fernando C Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.
- [57] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.
- [58] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [59] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.
- [60] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
- [61] Arash Vahdat, Karsten Kreis, and Jan Kautz. Score-based generative modeling in latent space. *Advances in neural information processing systems*, 34:11287–11302, 2021.
- [62] Michael Janner, Yilun Du, Joshua B Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.
- [63] Matthew Thomas Jackson, Michael Tryfan Matthews, Cong Lu, Benjamin Ellis, Shimon Whiteson, and Jakob Foerster. Policy-guided diffusion. *arXiv preprint arXiv:2404.06356*, 2024.
- [64] Cong Lu, Philip Ball, Yee Whye Teh, and Jack Parker-Holder. Synthetic experience replay. *Advances in Neural Information Processing Systems*, 36:46323–46344, 2023.
- [65] Zihan Ding, Amy Zhang, Yuandong Tian, and Qingqing Zheng. Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning. *arXiv preprint arXiv:2402.03570*, 2024.
- [66] Grace Luo, Lisa Dunlap, Dong Huk Park, Aleksander Holynski, and Trevor Darrell. Diffusion hyperfeatures: Searching through time and space for semantic correspondence. *Advances in Neural Information Processing Systems*, 36:47500–47510, 2023.
- [67] Zhongqi Yue, Jiankun Wang, Qianru Sun, Lei Ji, Eric I Chang, Hanwang Zhang, et al. Exploring diffusion time-steps for unsupervised representation learning. *arXiv preprint arXiv:2401.11430*, 2024.
- [68] Xingyi Yang and Xinchao Wang. Diffusion model as representation learner. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18938–18949, 2023.
- [69] Zijian Zhang, Zhou Zhao, and Zhijie Lin. Unsupervised representation learning from pre-trained diffusion probabilistic models. *Advances in neural information processing systems*, 35: 22117–22130, 2022.
- [70] Kushagra Pandey, Avideep Mukherjee, Piyush Rai, and Abhishek Kumar. Diffusevae: Efficient, controllable and high-fidelity generation from low-dimensional latents. *arXiv preprint arXiv:2201.00308*, 2022.
- [71] Beatrix MG Nielsen, Anders Christensen, Andrea Dittadi, and Ole Winther. Diffenc: Variational diffusion with a learned encoder. *arXiv preprint arXiv:2310.19789*, 2023.

- [72] Dmitry Baranchuk, Ivan Rubachev, Andrey Voynov, Valentin Khruikov, and Artem Babenko. Label-efficient semantic segmentation with diffusion models. *arXiv preprint arXiv:2112.03126*, 2021.
- [73] Weilai Xiang, Hongyu Yang, Di Huang, and Yunhong Wang. Denoising diffusion autoencoders are unified self-supervised learners. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15802–15812, 2023.
- [74] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.

A Technical appendices and supplementary material

A.1 JEPA as a Variational Information Bottleneck

We consider two observed views or two sequential observations, x_1 and x_2 , together with latent representations z_1 and z_2 . The assumed probabilistic graphical model is

$$x_1 \leftarrow z_1 \rightarrow z_2 \rightarrow x_2.$$

Under this model, the joint distribution factorizes as

$$p_\theta(x_1, x_2, z_1, z_2) = p(z_1) p_\theta(z_2 | z_1) p(x_1 | z_1) p(x_2 | z_2). \quad (11)$$

Here, $p_\theta(z_2 | z_1)$ plays the role of the JEPA predictor: it predicts the latent representation of the second view from the latent representation of the first view.

We introduce the amortized variational posterior

$$q_\phi(z_1, z_2 | x_1, x_2) = q_\phi(z_1 | x_1) q_\phi(z_2 | x_2). \quad (12)$$

For notational compactness, for a fixed pair (x_1, x_2) , define

$$q_1(z_1) := q_\phi(z_1 | x_1), \quad q_2(z_2) := q_\phi(z_2 | x_2), \quad q_{12}(z_1, z_2) := q_1(z_1) q_2(z_2).$$

Variational decomposition of the joint likelihood. We begin with the marginal log-likelihood of the two views. Since q_{12} is a normalized distribution over z_1, z_2 , we may write

$$\begin{aligned} \log p_\theta(x_1, x_2) &= \int q_{12}(z_1, z_2) \log p_\theta(x_1, x_2) dz_1 dz_2 \\ &= \int q_{12}(z_1, z_2) \log \frac{p_\theta(x_1, x_2, z_1, z_2)}{p_\theta(z_1, z_2 | x_1, x_2)} dz_1 dz_2 \\ &= \mathbb{E}_{q_{12}} \left[\log \frac{p_\theta(x_1, x_2, z_1, z_2)}{q_{12}(z_1, z_2)} \right] + D_{\text{KL}}(q_{12}(z_1, z_2) \| p_\theta(z_1, z_2 | x_1, x_2)). \end{aligned} \quad (13)$$

Define the posterior gap

$$\Delta_\theta(x_1, x_2) := D_{\text{KL}}(q_{12}(z_1, z_2) \| p_\theta(z_1, z_2 | x_1, x_2)). \quad (14)$$

Substituting the factorization in Eq. (11) into Eq. (13), we obtain

$$\begin{aligned} \log p_\theta(x_1, x_2) &= \mathbb{E}_{q_1} [\log p(x_1 | z_1)] + \mathbb{E}_{q_2} [\log p(x_2 | z_2)] \\ &\quad - \mathbb{E}_{q_1} [D_{\text{KL}}(q_2(z_2) \| p_\theta(z_2 | z_1))] - D_{\text{KL}}(q_1(z_1) \| p(z_1)) + \Delta_\theta(x_1, x_2). \end{aligned} \quad (15)$$

The first two terms are view-likelihood terms. The third term is the JEPA prediction term. The fourth term is a bottleneck regularizer on z_1 , and the final term is the posterior approximation gap.

Connection to mutual information. The mutual information between the two views is

$$I(X_1; X_2) = \mathbb{E}_{p(x_1, x_2)} [\log p(x_1, x_2) - \log p(x_1) - \log p(x_2)]. \quad (16)$$

Assuming the model marginal matches the population distribution, define the variational estimates

$$\hat{I}(X_1; Z_1) := \mathbb{E}_{p(x_1)q_\phi(z_1|x_1)} [\log p(x_1 | z_1) - \log p(x_1)], \quad (17)$$

$$\hat{I}(X_2; Z_2) := \mathbb{E}_{p(x_2)q_\phi(z_2|x_2)} [\log p(x_2 | z_2) - \log p(x_2)]. \quad (18)$$

Now define the JEPA prediction loss

$$\mathcal{L}_{\text{JEPA}} := \mathbb{E}_{p(x_1, x_2)q_\phi(z_1|x_1)} [D_{\text{KL}}(q_\phi(z_2 | x_2) \| p_\theta(z_2 | z_1))], \quad (19)$$

the bottleneck regularizer

$$\mathcal{R}_1 := \mathbb{E}_{p(x_1)} [D_{\text{KL}}(q_\phi(z_1 | x_1) \| p(z_1))], \quad (20)$$

and the expected posterior gap

$$\mathcal{G} := \mathbb{E}_{p(x_1, x_2)} [D_{\text{KL}}(q_\phi(z_1, z_2 | x_1, x_2) \| p_\theta(z_1, z_2 | x_1, x_2))]. \quad (21)$$

Then

$$I(X_1; X_2) = \hat{I}(X_1; Z_1) + \hat{I}(X_2; Z_2) - \mathcal{L}_{\text{JEPA}} - \mathcal{R}_1 + \mathcal{G}. \quad (22)$$

Equivalently,

$$-\mathcal{L}_{\text{JEPA}} = I(X_1; X_2) - \hat{I}(X_1; Z_1) - \hat{I}(X_2; Z_2) + \mathcal{R}_1 - \mathcal{G}. \quad (23)$$

Under the generative Markov chain $X_1 - Z_1 - Z_2 - X_2$, the data processing inequality gives

$$I(X_1; X_2) \leq I(Z_1; Z_2), \quad (24)$$

and therefore

$$-\mathcal{L}_{\text{JEPA}} \leq I(Z_1; Z_2) - \hat{I}(X_1; Z_1) - \hat{I}(X_2; Z_2) + \mathcal{R}_1 - \mathcal{G}. \quad (25)$$

A.2 Conditional Diffusion-JEPA as a Variational Information Bottleneck

We now extend the variational JEPA argument to a latent conditional diffusion predictor. The key difference from the one-step JEPA model is that the direct predictor $p_\theta(z_2 | z_1)$ is replaced by a conditional reverse diffusion trajectory that predicts the clean target latent z_2^0 from noise while conditioning on the clean context latent z_1^0 .

We consider two observed views x_1 and x_2 . The clean context latent is z_1^0 , and the target latent is represented by a diffusion path

$$z_2^{0:T} := (z_2^0, z_2^1, \dots, z_2^T),$$

where z_2^0 is the clean target latent and z_2^T is the maximally noised latent. The assumed graphical model is shown in fig. 3, where each reverse denoising transition is conditioned on z_1^0 . Equivalently, the joint distribution factorizes as

$$p_\psi(x_1, x_2, z_1^0, z_2^{0:T}) = p(z_1^0) p(x_1 | z_1^0) p(z_2^T) \prod_{t=1}^T p_\psi(z_2^{t-1} | z_2^t, z_1^0) p(x_2 | z_2^0). \quad (26)$$

Here,

$$p_\psi(z_2^{0:T} | z_1^0) := p(z_2^T) \prod_{t=1}^T p_\psi(z_2^{t-1} | z_2^t, z_1^0)$$

is the conditional reverse diffusion path. Therefore, the induced stochastic JEPA predictor from z_1^0 to z_2^0 is

$$p_\psi(z_2^0 | z_1^0) = \int p(z_2^T) \prod_{t=1}^T p_\psi(z_2^{t-1} | z_2^t, z_1^0) dz_2^{1:T}. \quad (27)$$

Thus, the conditional diffusion model is a multi-step stochastic JEPA predictor.

We introduce the amortized variational posterior

$$q_\varphi(z_1^0, z_2^{0:T} | x_1, x_2) = q_\varphi(z_1^0 | x_1) q_\varphi(z_2^0 | x_2) q(z_2^{1:T} | z_2^0), \quad (28)$$

where $q(z_2^{1:T} | z_2^0)$ is the fixed forward diffusion process. For notational compactness, for a fixed pair (x_1, x_2) , define

$$\begin{aligned} q_1(z_1^0) &:= q_\varphi(z_1^0 | x_1), \\ q_2^0(z_2^0) &:= q_\varphi(z_2^0 | x_2), \\ q_2^{0:T}(z_2^{0:T}) &:= q_2^0(z_2^0)q(z_2^{1:T} | z_2^0), \end{aligned}$$

and

$$q_{12}(z_1^0, z_2^{0:T}) := q_1(z_1^0)q_2^{0:T}(z_2^{0:T}).$$

Variational decomposition of the joint likelihood. We begin with the marginal log-likelihood of the two views. Since q_{12} is a normalized distribution over $z_1^0, z_2^{0:T}$, we may write

$$\begin{aligned} \log p_\psi(x_1, x_2) &= \int q_{12}(z_1^0, z_2^{0:T}) \log p_\psi(x_1, x_2) dz_1^0 dz_2^{0:T} \\ &= \int q_{12}(z_1^0, z_2^{0:T}) \log \frac{p_\psi(x_1, x_2, z_1^0, z_2^{0:T})}{p_\psi(z_1^0, z_2^{0:T} | x_1, x_2)} dz_1^0 dz_2^{0:T} \\ &= \mathbb{E}_{q_{12}} \left[\log \frac{p_\psi(x_1, x_2, z_1^0, z_2^{0:T})}{q_{12}(z_1^0, z_2^{0:T})} \right] + D_{\text{KL}}(q_{12}(z_1^0, z_2^{0:T}) \| p_\psi(z_1^0, z_2^{0:T} | x_1, x_2)). \end{aligned} \quad (29)$$

Define the posterior gap

$$\Delta_\psi(x_1, x_2) := D_{\text{KL}}(q_{12}(z_1^0, z_2^{0:T}) \| p_\psi(z_1^0, z_2^{0:T} | x_1, x_2)). \quad (30)$$

Substituting the factorization in Eq. (26) into Eq. (29), we obtain

$$\begin{aligned} \log p_\psi(x_1, x_2) &= \mathbb{E}_{q_{12}} \left[\log p(x_1 | z_1^0) + \log p(x_2 | z_2^0) + \log p(z_1^0) \right. \\ &\quad \left. + \log p(z_2^T) + \sum_{t=1}^T \log p_\psi(z_2^{t-1} | z_2^t, z_1^0) - \log q_1(z_1^0) - \log q_2^{0:T}(z_2^{0:T}) \right] \\ &\quad + \Delta_\psi(x_1, x_2). \end{aligned} \quad (31)$$

Now group the terms according to their roles. The view-likelihood terms are

$$\mathbb{E}_{q_{12}}[\log p(x_1 | z_1^0)] = \mathbb{E}_{q_1}[\log p(x_1 | z_1^0)]$$

and

$$\mathbb{E}_{q_{12}}[\log p(x_2 | z_2^0)] = \mathbb{E}_{q_2^0}[\log p(x_2 | z_2^0)].$$

The context-prior terms satisfy

$$\mathbb{E}_{q_{12}}[\log p(z_1^0) - \log q_1(z_1^0)] = -D_{\text{KL}}(q_1(z_1^0) \| p(z_1^0)). \quad (32)$$

The target-path terms satisfy

$$\begin{aligned} \mathbb{E}_{q_{12}} \left[\log p(z_2^T) + \sum_{t=1}^T \log p_\psi(z_2^{t-1} | z_2^t, z_1^0) - \log q_2^{0:T}(z_2^{0:T}) \right] \\ = -\mathbb{E}_{q_1} [D_{\text{KL}}(q_2^{0:T}(z_2^{0:T}) \| p_\psi(z_2^{0:T} | z_1^0))]. \end{aligned} \quad (33)$$

Therefore,

$$\begin{aligned} \log p_\psi(x_1, x_2) &= \mathbb{E}_{q_1}[\log p(x_1 | z_1^0)] + \mathbb{E}_{q_2^0}[\log p(x_2 | z_2^0)] \\ &\quad - \mathbb{E}_{q_1} [D_{\text{KL}}(q_2^{0:T}(z_2^{0:T}) \| p_\psi(z_2^{0:T} | z_1^0))] \\ &\quad - D_{\text{KL}}(q_1(z_1^0) \| p(z_1^0)) + \Delta_\psi(x_1, x_2). \end{aligned} \quad (34)$$

The first two terms are view-likelihood terms. The third term is the conditional diffusion-JEPA prediction term. The fourth term is a bottleneck regularizer on the clean context latent z_1^0 , and the final term is the posterior approximation gap.

Define the instance-level conditional diffusion path loss

$$\mathcal{L}_{\text{CDJ}}^{\text{path}}(x_1, x_2) := \mathbb{E}_{q_1} [D_{\text{KL}}(q_2^{0:T}(z_2^{0:T}) \parallel p_\psi(z_2^{0:T} \mid z_1^0))]. \quad (35)$$

This is the direct analogue of the one-step JEPA prediction loss $\mathbb{E}_{q_1} [D_{\text{KL}}(q_2(z_2) \parallel p_\theta(z_2 \mid z_1))]$, except that the predicted object is now the entire reverse diffusion path $z_2^{0:T}$.

To see the diffusion terms explicitly, write

$$\mathcal{L}_{\text{CDJ}}^{\text{path}}(x_1, x_2) = \mathbb{E}_{q_1 q_2^{0:T}} \left[\log q_2^{0:T}(z_2^{0:T}) - \log p(z_2^T) - \sum_{t=1}^T \log p_\psi(z_2^{t-1} \mid z_2^t, z_1^0) \right]. \quad (36)$$

Since

$$q_2^{0:T}(z_2^{0:T}) = q_2^0(z_2^0) q(z_2^{1:T} \mid z_2^0),$$

we may also write

$$\begin{aligned} \mathcal{L}_{\text{CDJ}}^{\text{path}}(x_1, x_2) = \mathbb{E}_{q_1 q_2^0 q(z_2^{1:T} \mid z_2^0)} & \left[\log q_2^0(z_2^0) + \log q(z_2^{1:T} \mid z_2^0) \right. \\ & \left. - \log p(z_2^T) - \sum_{t=1}^T \log p_\psi(z_2^{t-1} \mid z_2^t, z_1^0) \right]. \end{aligned} \quad (37)$$

For a Markov forward diffusion process, this path loss admits the usual DDPM decomposition. Using

$$q(z_2^{1:T} \mid z_2^0) = q(z_2^T \mid z_2^0) \prod_{t=2}^T q(z_2^{t-1} \mid z_2^t, z_2^0),$$

we obtain

$$\begin{aligned} \mathcal{L}_{\text{CDJ}}^{\text{path}}(x_1, x_2) = & -H(q_2^0) + \mathbb{E}_{q_2^0} [D_{\text{KL}}(q(z_2^T \mid z_2^0) \parallel p(z_2^T))] \\ & + \mathbb{E}_{q_1 q_2^0 q(z_2^1 \mid z_2^0)} [-\log p_\psi(z_2^1 \mid z_2^2, z_1^0)] \\ & + \sum_{t=2}^T \mathbb{E}_{q_1 q_2^0 q(z_2^t \mid z_2^0)} [D_{\text{KL}}(q(z_2^{t-1} \mid z_2^t, z_2^0) \parallel p_\psi(z_2^{t-1} \mid z_2^t, z_1^0))]. \end{aligned} \quad (38)$$

The first term is the entropy of the target encoder distribution, the second term is the terminal prior-matching term, the third term is the endpoint reconstruction term for z_2^0 , and the remaining terms are the conditional denoising KLs.

Connection to mutual information. The mutual information between the two views is

$$I(X_1; X_2) = \mathbb{E}_{p(x_1, x_2)} [\log p(x_1, x_2) - \log p(x_1) - \log p(x_2)]. \quad (39)$$

Assuming the model marginal matches the population distribution, define the variational estimates

$$\hat{I}(X_1; Z_1^0) := \mathbb{E}_{p(x_1) q_\varphi(z_1^0 \mid x_1)} [\log p(x_1 \mid z_1^0) - \log p(x_1)], \quad (40)$$

$$\hat{I}(X_2; Z_2^0) := \mathbb{E}_{p(x_2) q_\varphi(z_2^0 \mid x_2)} [\log p(x_2 \mid z_2^0) - \log p(x_2)]. \quad (41)$$

Now define the population-level conditional diffusion-JEPA path loss

$$\mathcal{L}_{\text{CDJ}}^{\text{path}} := \mathbb{E}_{p(x_1, x_2)} [\mathcal{L}_{\text{CDJ}}^{\text{path}}(x_1, x_2)], \quad (42)$$

the bottleneck regularizer

$$\mathcal{R}_1 := \mathbb{E}_{p(x_1)} [D_{\text{KL}}(q_\varphi(z_1^0 \mid x_1) \parallel p(z_1^0))], \quad (43)$$

and the expected posterior gap

$$\mathcal{G} := \mathbb{E}_{p(x_1, x_2)} [D_{\text{KL}}(q_\varphi(z_1^0, z_2^{0:T} \mid x_1, x_2) \parallel p_\psi(z_1^0, z_2^{0:T} \mid x_1, x_2))]. \quad (44)$$

Taking the expectation of Eq. (34) over $p(x_1, x_2)$ and subtracting $\log p(x_1) + \log p(x_2)$ gives

$$I(X_1; X_2) = \hat{I}(X_1; Z_1^0) + \hat{I}(X_2; Z_2^0) - \mathcal{L}_{\text{CDJ}}^{\text{path}} - \mathcal{R}_1 + \mathcal{G}. \quad (45)$$

Equivalently,

$$-\mathcal{L}_{\text{CDJ}}^{\text{path}} = I(X_1; X_2) - \hat{I}(X_1; Z_1^0) - \hat{I}(X_2; Z_2^0) + \mathcal{R}_1 - \mathcal{G}. \quad (46)$$

After marginalizing the intermediate diffusion variables $z_2^{1:T}$, the model induces the clean-latent Markov structure

$$X_1 - Z_1^0 - Z_2^0 - X_2.$$

Therefore, by the data processing inequality,

$$I(X_1; X_2) \leq I(Z_1^0; Z_2^0). \quad (47)$$

Substituting this into Eq. (46) gives

$$-\mathcal{L}_{\text{CDJ}}^{\text{path}} \leq I(Z_1^0; Z_2^0) - \hat{I}(X_1; Z_1^0) - \hat{I}(X_2; Z_2^0) + \mathcal{R}_1 - \mathcal{G}. \quad (48)$$

Finally, if one optimizes only the DDPM-style learnable denoising part of the path objective, define

$$\begin{aligned} \mathcal{L}^0 &:= \mathbb{E}_{p(x_1, x_2)} \mathbb{E}_{q_1 q_2^0 q(z_2^1 | z_2^0)} [-\log p_\psi(z_2^0 | z_2^1, z_1^0)], \\ \mathcal{L}_{\text{CDJ}}^{\text{den}} &:= \mathcal{L}^0 + \mathbb{E}_{p(x_1, x_2)} \sum_{t=2}^T \mathbb{E}_{q_1 q_2^0 q(z_2^t | z_2^0)} [D_{\text{KL}}(q(z_2^{t-1} | z_2^t, z_2^0) \| p_\psi(z_2^{t-1} | z_2^t, z_1^0))]. \end{aligned} \quad (49)$$

Then

$$\mathcal{L}_{\text{CDJ}}^{\text{path}} = \mathcal{C}_2 + \mathcal{R}_{2,T} + \mathcal{L}_{\text{CDJ}}^{\text{den}}, \quad (50)$$

where

$$\mathcal{R}_{2,T} := \mathbb{E}_{p(x_2) q_\varphi(z_2^0 | x_2)} [D_{\text{KL}}(q(z_2^T | z_2^0) \| p(z_2^T))] \quad (51)$$

is the terminal prior-matching term, while

$$\mathcal{C}_2 := \mathbb{E}_{p(x_2)} [-H(q_\varphi(z_2^0 | x_2))] = \mathbb{E}_{p(x_2) q_\varphi(z_2^0 | x_2)} [\log q_\varphi(z_2^0 | x_2)] \quad (52)$$

is the ψ -independent remainder. Substituting Eq. (50) into Eq. (48) yields

$$\begin{aligned} -\mathcal{L}_{\text{CDJ}}^{\text{den}} &\leq I(Z_1^0; Z_2^0) - \hat{I}(X_1; Z_1^0) - \hat{I}(X_2; Z_2^0) \\ &\quad + \mathcal{R}_1 + \mathcal{R}_{2,T} - \mathcal{G} + \mathcal{C}_2. \end{aligned} \quad (53)$$

Therefore, even when training is written only as a conditional denoising objective, the conditional diffusion-JEPA loss exhibits the same variational information-bottleneck structure as the one-step JEPA loss, up to terminal diffusion regularization, endpoint constants, and the variational posterior gap.

A.2.1 Remark on optimizing the bottleneck regularizer.

The decomposition above shows that the exact variational information-bottleneck objective is

$$\min_{\varphi, \psi} \mathcal{L}_{\text{CDJ}}^{\text{path}} + \mathcal{R}_1, \quad (54)$$

or, more generally,

$$\min_{\varphi, \psi} \mathcal{L}_{\text{CDJ}}^{\text{path}} + \beta \mathcal{R}_1. \quad (55)$$

In our experiments, we do not directly optimize the exact KL regularizer $\mathcal{R}_1$. Accordingly, the most precise interpretation of JEDI is that it optimizes the conditional diffusion prediction term that appears inside the variational information-bottleneck decomposition, rather than exactly optimizing the full variational information-bottleneck objective. We therefore present the bottleneck view as a theoretical justification and design lens for the current method, while treating explicit optimization of $\mathcal{R}_1$ as a promising extension.

Relation to VICReg and LeJEPa [41, 40] The bottleneck regularizer $\mathcal{R}_1$ also clarifies the connection between our variational view and recent explicit regularization methods for collapse-free joint embedding learning. We do not incorporate such regularizers here, since our empirical study focuses on conventional JEPa training, but the decomposition reveals a close conceptual link.

By the standard decomposition,

$$\mathcal{R}_1 = \mathbb{E}_{p(x_1)} D_{\text{KL}}(q_\varphi(z_1^0 | x_1) \| p(z_1^0)) = I_q(X_1; Z_1^0) + D_{\text{KL}}(q_\varphi(z_1^0) \| p(z_1^0)). \quad (56)$$

The first term is the information-bottleneck component, which explicitly penalizes the information retained by the context representation. The second term is an aggregate distribution-matching regularizer. When $p(z_1^0) = \mathcal{N}(0, I)$, this aggregate term encourages the representation distribution to be centered, non-collapsed, and isotropic. This is closely related in spirit to the variance and covariance regularizers used in VICReg, which maintain per-dimension variance and decorrelate embedding coordinates, and to the SIGReg regularizer used in LeJEPa, which directly encourages embeddings to match an isotropic Gaussian distribution.

Thus, $\mathcal{R}_1$ can be viewed as a variational counterpart to these explicit representation regularizers, with an additional information-bottleneck interpretation.

A.2.2 Remark on $\mathcal{C}_2$ remainder term

With the exact DDPM KL decomposition, the remainder term has the explicit form

$$\mathcal{C}_2(x_2) := -H(q_\varphi(z_2^0 | x_2)) = \mathbb{E}_{q_\varphi(z_2^0 | x_2)} [\log q_\varphi(z_2^0 | x_2)]. \quad (57)$$

At the population level,

$$\mathcal{C}_2 = \mathbb{E}_{p(x_2)} [\mathbb{E}_{q_\varphi(z_2^0 | x_2)} \log q_\varphi(z_2^0 | x_2)]. \quad (58)$$

Thus,

$$\mathcal{L}_{\text{CDJ}}^{\text{path}} = \mathcal{C}_2 + \mathcal{R}_{2,T} + \mathcal{L}_{\text{CDJ}}^{\text{den}}. \quad (59)$$

When $q_\varphi(z_2^0 | x_2)$ has fixed entropy, for example under a deterministic or fixed-variance target encoder with stop-gradient, $\mathcal{C}_2$ is constant with respect to the conditional reverse parameters ψ . If the target encoder distribution is learnable through this objective, however, $\mathcal{C}_2$ may depend on φ and should not be treated as a global constant.

A.3 JEDI Algorithm

Green font refers to changes as compared to pixel-based diffusion world model baseline [7]

Algorithm 1 JEDI Training

```

1: procedure TRAINING_LOOP
2:   for epochs do
3:     collect_experience(steps_collect)
4:     for steps_diffusion_model do
5:       update_latent_diffusion_model()
6:     end for
7:     for steps_reward_end_model do
8:       update_reward_end_model()
9:     end for
10:    for steps_actor_critic do
11:      update_actor_critic()
12:    end for
13:  end for
14: end procedure

15: procedure COLLECT_EXPERIENCE( $n$ )
16:    $x_0 \leftarrow \text{env.reset}()$ 
17:   for  $t = 0$  to  $n - 1$  do
18:     Sample  $a_t \sim \pi_\omega(a_t|z_t^0)q_\phi(z_t^0|x_t)$  ▷ derive  $z_t^0$  using JEDI encoder  $\mathbf{E}_\phi$  before sampling action
19:      $x_{t+1}, r_t, d_t \leftarrow \text{env.step}(a_t)$ 
20:      $\mathcal{D} \leftarrow \mathcal{D} \cup \{x_t, a_t, r_t, d_t\}$ 
21:     if  $d_t = \text{true}$  then
22:        $x_{t+1} \leftarrow \text{env.reset}()$ 
23:     end if
24:   end for
25: end procedure

26: procedure UPDATE_LATENT_DIFFUSION_MODEL
27:   Sample sequence  $(x_{t-L+1}, a_{t-L+1}, \dots, x_t, a_t, x_{t+1}) \sim \mathcal{D}$ 
28:   Sample  $\log(\sigma) \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$  ▷ log-normal sigma distribution from EDM
29:   Define  $\tau := \sigma$  ▷ default identity schedule from EDM
30:   compute  $[z_{t-L+1}^0, \dots, z_t^0, z_{t+1}^0] = C(\mathbf{E}_\phi([x_{t-L+1}, \dots, x_t, x_{t+1}]))$  ▷ derive the clamped latents
31:   derive  $z_{t+1}^0 = \text{sg}(z_{t+1}^0)$  ▷ detach gradients from the target latent
32:   Sample  $z_{t+1}^\tau \sim \mathcal{N}(z_{t+1}^0, \sigma^2 \mathbf{I})$  ▷ add independent Gaussian noise
33:   Compute  $\hat{z}_{t+1}^0 = \mathbf{D}_\theta(z_{t+1}^\tau, \tau, z_{t-L+1}^0, a_{t-L+1}, \dots, z_t^0, a_t)$ 
34:   Compute reconstruction loss  $\mathcal{L}(\theta) = \|\hat{z}_{t+1}^0 - z_{t+1}^0\|^2$ 
35:    $rs \sim \text{Uniform}\{\text{True}, \text{False}\}$ 
36:   if  $rs = \text{True}$  then ▷ random switch between  $\hat{z}_{t+1}^0$  or  $z_{t+1}^0$  as  $\mathbf{D}_\theta$  input for loss at  $t + 1$ 
37:     Cache  $\hat{z}_{t+1}^0$  as input to  $\mathbf{D}_\theta$  for UPDATE_LATENT_DIFFUSION_MODEL at  $t + 1$ 
38:   end if
39:   Update  $\mathbf{D}_\theta$ 
40: end procedure

41: procedure UPDATE_REWARD_END_MODEL
42:   Sample indexes  $\mathcal{I} = \{t, \dots, t + L + H - 1\}$  ▷ burn-in + imagination horizon
43:   Sample sequence  $(x_i, a_i, r_i, d_i)_{i \in \mathcal{I}} \sim \mathcal{D}$ 
44:   Compute  $(z_i^0)_{i \in \mathcal{I}} = C(\mathbf{E}_\phi((x_i)_{i \in \mathcal{I}}))$  ▷ derive the clamped latents
45:   Initialize  $h = c = 0$  ▷ LSTM hidden and cell states
46:   for  $i \in \mathcal{I}$  do
47:     Compute  $\hat{r}_i, \hat{d}_i, h, c = R_\psi(z_i^0, a_i, h, c)$ 
48:   end for
49:   Compute  $\mathcal{L}(\psi) = \sum_{i \in \mathcal{I}} \text{CE}(\hat{r}_i, \text{sign}(r_i)) + \text{CE}(\hat{d}_i, d_i)$  ▷ CE: cross-entropy loss
50:   Update  $R_\psi$ 
51: end procedure

52: procedure UPDATE_ACTOR_CRITIC
53:   Sample initial buffer  $(x_{t-L+1}, a_{t-L+1}, \dots, x_t) \sim \mathcal{D}$ 
54:   Compute  $[z_{t-L+1}^0, \dots, z_t^0] = C(\mathbf{E}_\phi([x_{t-L+1}, \dots, x_t]))$  ▷ derive the clamped latents
55:   Burn-in buffer with  $R_\psi, \pi_\omega$  and  $V_\omega$  to initialize LSTM states
56:   for  $i = t$  to  $t + H - 1$  do
57:     Sample  $a_i \sim \pi_\omega(a_i|z_i^0)$ 
58:     Sample reward  $r_i$  and termination  $d_i$  with  $R_\psi$ 
59:     Sample next observation  $z_{i+1}^0$  by simulating reverse diffusion process with  $\mathbf{D}_\theta$ 
60:   end for
61:   Compute  $V_\omega(z_i^0)$  for  $i = t, \dots, t + H$ 
62:   Compute RL losses  $\mathcal{L}_V(\omega)$  and  $\mathcal{L}_\pi(\omega)$ 
63:   Update  $\pi_\omega$  and  $V_\omega$ 
64: end procedure

```

B Changes Compared to Baseline and Hyperparameters

We used the DIAMOND library [7] (MIT License) for our implementation. The main difference between JEDI World Model and DIAMOND is the transplant of the DIAMOND’s RL encoder onto the JEDI World Model and the techniques implemented to facilitate end-to-end latent diffusion mentioned in section 3. The only network architectural changes are practical: (1) changing the original RL encoder to downsample to $z_t \in [-3, 3]^{16 \times 8 \times 8}$, (2) modifying the diffusion UNET [74] into a single layer without downsampling, (3) and removing the downsampling in the Reward/Termination Model’s encoder. The only hyperparameter changes were increasing the diffusion model’s warm-up learning steps to 1000 and sigma of input data to be 1.

B.1 Benchmark and environment settings

Atari100k. We follow the standard Atari100k protocol used by prior online world-model work and the IRIS evaluation codebase. In particular, each run uses the 26-game Atari100k benchmark with a total budget of 100k environment interactions, and we report results over 5 seeds per task as described in section 4.1. Observations are RGB frames resized to 64×64 , matching the image interface used by the JEDI encoder and world model. We use the standard Atari v4 setup with frame skip 4, which matches the protocol used in prior Atari100k world-model evaluations.

Evaluation metrics. Following Agarwal et al. [44], we compute aggregate Atari100k metrics from human-normalized scores (HNS). For a game with agent score S_{agent} , random-policy score S_{random} , and human score S_{human} , the human-normalized score is

$$\text{HNS} = \frac{S_{\text{agent}} - S_{\text{random}}}{S_{\text{human}} - S_{\text{random}}}.$$

An HNS of 1 corresponds to human-level performance. “Mean” refers to the arithmetic mean of HNS values across the benchmark. “IQM” (interquartile mean) is the mean of the middle 50% of HNS values across game seed runs, which makes it more robust to outliers than the plain mean. “Optimality Gap” measures how far performance remains below human level, and is reported as the average gap to the threshold $\text{HNS} = 1$, i.e., $\mathbb{E}[\max(1 - \text{HNS}, 0)]$, so lower is better.

Craftium. For Craftium, we evaluate on four tasks with 3 seeds per task, as described in section 4.1. We use standard visual observations with RGB images resized to 64×64 , so that the same encoder and latent world-model interface used on Atari can be applied directly in the 3D setting. We use 300 training steps and 200 collection steps per epoch. We report the training returns. We leverage the symlog reward prediction used in HI [8] for the continuous reward task, Speleo. We also follow HI and use 30k environment steps for SmallRoom-v0 and 100k environment steps for the other 3 environments

B.2 Third-party assets and licenses

We explicitly credit the third-party assets used in this work and follow their stated licenses and terms of use. Our implementation builds on the DIAMOND library [7], which is released under the MIT License. For Atari100k evaluation, we used the publicly available IRIS codebase associated with Micheli et al. [46], which is released under the GNU General Public License v3.0. For the Craftium experiments, we used the Craftium environment [35]; its repository states that the project inherits the Luant/Minetest licensing scheme, with code under the MIT License and game content under CC-BY-SA 3.0. These assets are cited in the bibliography and used in accordance with their repository-stated licenses.

Table 2: Model architecture details for JEDI. Green font refers to changes as compared to pixel-based diffusion world model baseline [7]. Downsampling layers refers to the Maxpool operation that downsamples the input height and width by a factor of 2.

Hyperparameter	Value
Latent Diffusion Dynamics Model (D_θ)	
State conditioning mechanism	Frame stacking
Action conditioning mechanism	Adaptive Group Normalization
Diffusion time conditioning mechanism	Adaptive Group Normalization
Residual blocks layers	[1]
Residual blocks channels	[160]
UNET Downsampling	NIL
Residual blocks conditioning dimension	256
Sigma data (for preconditioning)	1
World Model Encoder (E_ϕ)	
Encoder blocks layers	[1, 1, 1, 1]
Encoder blocks channels	[32, 32, 32, 16]
Encoder Downsampling layers	[1,1,1,0]
Encoder tanh clamp factor	3
Reward/Termination Model (R_ψ)	
Action conditioning mechanisms	Adaptive Group Normalization
Residual blocks layers	[2, 2, 2, 2]
Residual blocks channels	[32, 32, 32, 32]
Encoder Downsampling layers	NIL
Residual blocks conditioning dimension	128
LSTM dimension	512
Actor-Critic Model (π_ω and V_ω)	
Encoder	NIL
LSTM dimension	512

Table 3: Training hyperparameters for JEDI. Green font refers to changes as compared to pixel-based diffusion world model baseline [7].

Hyperparameter	Value
Training loop	
Number of epochs	1000
Training steps per epoch	400
Denoiser Batch size	32/64
Other Batch size	32
Environment steps per epoch	100
Epsilon (greedy) for collection	0.01
RL hyperparameters	
Imagination horizon (H)	15
Discount factor (γ)	0.985
Entropy weight (η)	0.001
λ -returns coefficient (λ)	0.95
Sequence construction during training	
For D_θ , number of conditioning observations and actions (L)	4
For R_ψ , burn-in length (B_R), set to L in practice	4
For R_ψ , training sequence length ($B_R + H$)	19
For π_ϕ and V_ϕ , burn-in length ($B_{\pi,V}$), set to L in practice	4
Optimization	
Optimizer	AdamW
Learning rate	1e-4
Epsilon	1e-8
Weight decay (D_θ)	1e-2
Weight decay (R_ψ)	1e-2
Weight decay (π_ϕ and V_ϕ)	0
Learning rate Warm-up steps (D_θ)	1e3
Learning rate Warm-up steps (R_ψ)	1e2
Learning rate Warm-up steps (π_ω and V_ω)	1e2
Learning rate scale factor for E_ϕ	0.3
Diffusion Sampling	
Method	Euler
Number of steps	3
S churn	1 (only for stochastic experiments)
Environment	
Image observation dimensions	$64 \times 64 \times 3$
Action space	Discrete (up to 18 actions)
Frameskip	4
Frameskip for Stochastic Experiments	[2, 6]
Max noop	30
Termination on life loss	True
Reward clipping	{-1, 0, 1}

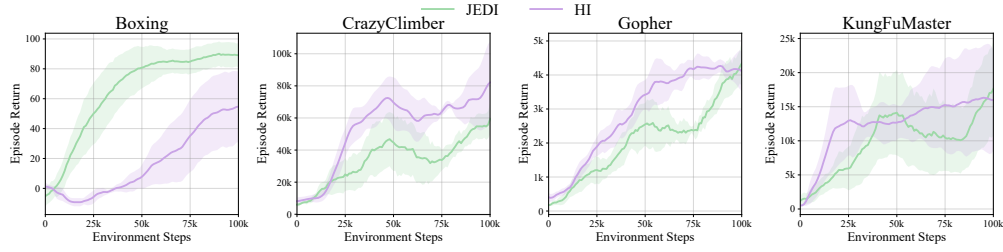


Figure 14: JEDI versus HI training curves on Atari100k

C Action Space and Shooter Tasks

Table 4: Atari100k tasks with number of actions and shooter-task label.

Task	Num Actions	Shooter Task?
<i>Boxing</i>	18	✗
<i>Krull</i>	18	✗
<i>CrazyClimber</i>	9	✗
<i>Gopher</i>	8	✗
<i>RoadRunner</i>	18	✗
<i>Jamesbond</i>	18	✓
<i>Assault</i>	7	✓
<i>Breakout</i>	4	✗
<i>KungFuMaster</i>	14	✗
<i>Pong</i>	6	✗
<i>Kangaroo</i>	18	✗
<i>UpNDown</i>	6	✗
<i>Freeway</i>	3	✗
<i>BankHeist</i>	18	✓
<i>DemonAttack</i>	6	✓
<i>Hero</i>	18	✓
<i>BattleZone</i>	18	✓
<i>Frostbite</i>	18	✗
<i>Qbert</i>	6	✗
<i>MsPacman</i>	9	✗
<i>Asterix</i>	9	✗
<i>ChopperCommand</i>	18	✓
<i>Amidar</i>	10	✗
<i>Alien</i>	18	✓
<i>PrivateEye</i>	18	✗
<i>Seaquest</i>	18	✓

D Supplementary Results

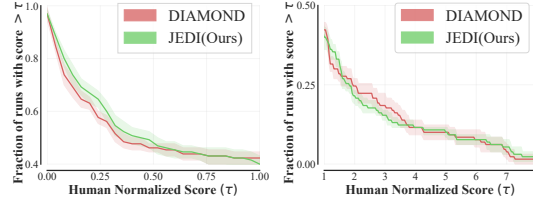


Figure 15: HNS performance profile. Left: low-HNS regime. Right: high-HNS regime. JEDI wins disproportionately often in low-HNS regimes, while DIAMOND’s largest relative wins occur in higher-HNS regimes.

Table 5: Atari100k overall performance. Bold refers to SOTA. We report results for batch size 32 and 64. Green score means it is the higher score and we take that result. We retrieve the updated results of STORM from [45]. TWISTER does not publish seed-level results, so we cannot compute its exact IQM or optimality gap. JEDI achieves SOTA optimality gap and IQM and runner-up performance in number of super-human games. JEDI achieves SOTA results in 7 games.

Game	Random	Human	TWM	IRIS	DreamerV3	STORM	DIAMOND	TWISTER	JEDI32/JEDI64	JEDI(Ours)
Alien	227.8	7127.7	674.6	420.0	1078.9	748.2	744.1	970.0	1090.1/801.6	1090.1
Amidar	5.8	1719.5	121.8	143.0	141.2	144.0	225.8	184.0	114.3/120.2	120.2
Assault	222.4	742.0	682.6	1524.4	669.7	1376.7	1526.4	721.0	1394.5/1388.3	1394.5
Asterix	210.0	8503.3	1116.6	853.6	946.4	1318.5	3698.5	1306.0	2331.0/2626.3	2626.3
BankHeist	14.2	753.1	466.7	53.1	622.7	990.0	19.7	942.0	926.1/653.3	926.1
BattleZone	2360.0	37187.5	5068.0	13074.0	12400.0	5830.0	4702.0	9920.0	16084.0/9056.0	16084.0
Boxing	0.1	12.1	77.5	70.1	73.8	81.2	86.9	88.0	91.6/86.8	91.6
Breakout	1.7	30.5	20.0	83.7	27.9	41.0	132.5	35.0	155.6/128.3	155.6
ChopperCommand	811.0	7387.8	1697.4	1565.0	411.7	1644.0	1369.8	910.0	1392.4/1589.2	1589.2
CrazyClimber	10780.5	35829.4	71820.4	59324.2	84880.0	79196.0	99167.8	81880.0	64786.2/69585.8	69585.8
DemonAttack	152.1	1971.0	350.2	2034.4	433.7	324.6	288.1	289.0	2047.2/1933.6	2047.2
Freeway	0.0	29.6	24.3	31.1	0.0	0.0	33.3	32.0	6.3/0.0	6.3
Frostbite	65.2	4334.7	1475.6	259.1	945.1	365.9	274.1	305.0	263.3/263.3	263.3
Gopher	257.6	2412.5	1674.8	2236.1	5754.3	5307.2	5897.9	22234.0	4155.3/4010.6	4155.3
Hero	1027.0	30826.4	7254.0	7037.4	11145.2	11434.1	5621.8	8773.0	7442.9/6861.1	7442.9
Jamesbond	29.0	302.8	362.4	462.7	480.4	408.0	427.4	573.0	460.5/448.2	460.5
Kangaroo	52.0	3035.0	1240.0	838.2	3790.0	3512.0	5382.2	6016.0	736.0/962.8	962.8
Krull	1598.0	2665.5	6349.2	6616.4	7921.5	6522.2	8610.1	8839.0	8499.4/8676.9	8676.9
KungFuMaster	258.5	22736.3	24554.6	21759.8	24210.0	20046.0	18713.6	23442.0	12725.2/21340.6	21340.6
MsPacman	307.3	6951.6	1588.4	999.1	1358.4	1489.5	1958.2	2206.0	1075.9/1024.4	1075.9
Pong	-20.7	14.6	18.8	14.6	18.9	18.4	20.4	20.0	18.2/18.4	18.4
PrivateEye	24.9	69571.3	86.6	100.0	1081.4	100.0	114.3	1608.0	95.2/90.5	95.2
Qbert	163.9	13455.0	3330.8	745.7	3291.3	2910.5	4499.3	3197.0	3606.1/3447.1	3606.1
RoadRunner	11.5	7845.0	9109.0	9614.6	14995.0	14841.0	20673.2	17832.0	14690.0/25736.6	25736.6
Seaquest	68.4	42054.7	774.4	661.3	610.0	557.4	551.2	532.0	1103.7/989.6	1103.7
UpNDown	533.4	11693.2	15981.7	3546.2	7981.7	6127.9	3856.3	7068.0	4191.1/4271.4	4271.4
#SOTA (↑)	0	N/A	4	0	0	2	7	6	-	7
#Superhuman (↑)	0	N/A	8	10	9	11	11	12	11/9	11
Mean (↑)	0.000	1.000	0.956	1.046	1.134	1.142	1.459	1.621	1.361/1.358	1.450
IQM (↑)	0.000	1.000	0.459	0.501	0.504	0.557	0.641	-	0.609/0.618	0.688
Optimality Gap (↓)	1.000	0.000	0.513	0.512	0.500	0.489	0.480	-	0.480/0.488	0.460